Rakamin Academy

kimia farma

Project Based Internship

SQL Operation Part 2

Basic Table Operation
Querying, Sorting, and Filtering

Daftar Isi

A. Basic Table Operation 3

1. SELECT 4

2. SELECT DISTINCT 4

3. ORDER BY 5

4. LIMIT 7

B. QUERYING, SORTING & FILTERING 7

1. WHERE 7

References 11

A. *Basic Table Operation*

Sebelum kita mulai menulis kueri, kita harus mengetahui simbol-simbol, penggantian nama, dan fungsi-fungsi SQL.

1. *: Tanda bintang adalah singkatan dari ALL. Jika ingin memilih semua kolom dalam tabel, tidak perlu mengetikkan semua nama kolom, cukup ketik *.
2. Ganti nama / Aliasing Bila ingin menyingkat nama tabel atau mengganti nama kolom, menambahkan nama baru setelah spasi atau *as nama alias*.
3. *Functions*: Terkadang kita tidak menginginkan data mentah, kita ingin melakukan operasi sederhana pada data tersebut. Fungsi yang umum digunakan adalah MAX/MIN/AVG/SUM/COUNT.

Dalam demonstrasi berikut, kita akan menggunakan tabel ini sebagai contoh. **Tabel Customers**, yang mencatat informasi pelanggan. Tabel ini memiliki 4 fitur - ID, Nama, Kota, Usia, dan ada 6 pelanggan dalam tabel ini. Selanjutnya, kunci utamanya adalah ID karena Anda hanya dapat menggunakan ID untuk mengidentifikasi setiap baris secara unik.

customers

ID	Name	City	Age
1	Lily	Melbourne	24
2	John	Chicago	55
3	Emily	Paris	17
4	Joyce	Tokyo	40
5	John	London	32
6	Adam	Paris	65

1. SELECT

Yang pertama adalah pernyataan paling dasar dalam SQL. Secara spesifik, sintaksnya adalah ***SELECT columns FROM table;***. Kita dapat memilih satu atau beberapa kolom sekaligus.

- ***SELECT Age FROM customers;***

	Age
0	24
1	55
2	17
3	40
4	32
5	65

- ***SELECT City, Age FROM customers;***

	City	Age
0	Melbourne	24
1	Chicago	55
2	Paris	17
3	Tokyo	40
4	London	32
5	Paris	65

2. SELECT DISTINCT

Untuk menghindari duplikasi, kita cukup menambahkan kata kunci DISTINCT sebelum nama kolom. Untuk contoh di atas, kueri menjadi

SELECT DISTINCT City FROM customers;

yang menghapus nilai duplikasi dan mengembalikan

	City
0	Melbourne
1	Chicago
2	Paris
3	Tokyo
4	London

3. ORDER BY

Selanjutnya, kita dapat mengurutkan kerangka data berdasarkan satu atau beberapa kolom. Katakanlah kita menggunakan 2 kolom untuk mengurutkan tabel, yaitu,

SELECT * FROM customers ORDER BY City, Age;

Tabel akan diurutkan berdasarkan kolom pertama ***CIty***; jika beberapa baris memiliki nilai yang sama di kolom pertama, mereka diurutkan berdasarkan kolom kedua ***Age***. Oleh karena itu, jika nilai pada kolom pertama unik, kolom kedua TIDAK digunakan sama sekali.

Kita selanjutnya dapat memutuskan apakah akan mengurutkan data secara menaik atau menurun dengan menambahkan **ASC** atau **DESC**. Secara

default, tabel diurutkan secara menaik. Namun, kita selalu dapat menentukan kata kunci untuk lebih memperjelas tujuan kita, yaitu,

SELECT * FROM customers ORDER BY City ASC, Age ASC;

, dan mengembalikan

	ID	Name	City	Age
0	2	John	Chicago	55
1	5	John	London	32
2	1	Lily	Melbourne	24
3	3	Emily	Paris	17
4	6	Adam	Paris	65
5	4	Joyce	Tokyo	40

Khususnya, kolom ***City*** dicantumkan dalam urutan Abjad karena nilai dalam kolom ini adalah string/karakter. Untuk data dengan nilai yang sama di ***City*** (misalnya, baris 3 dan 4 tinggal di Paris), mereka diurutkan berdasarkan kolom kedua Umur dalam urutan Numerik karena nilainya adalah bilangan bulat.

Bagaimana jika kita ingin mengurutkan tabel secara menurun berdasarkan Usia?

Query-nya adalah ***SELECT * FROM customers ORDER BY Age DESC;***

4. LIMIT

Ketika kita hanya ingin melihat n baris pertama dari data, kita tambahkan LIMIT n. Mengikuti contoh sebelumnya, kita ingin menemukan orang tertua, sehingga kueri menjadi

SELECT * FROM customers ORDER BY Age DESC LIMIT 1;

(karena kita hanya ingin 1 baris pertama) dan hasilnya adalah

	ID	Name	City	Age
0	6	Adam	Paris	65

B. QUERYING, SORTING & FILTERING

1. WHERE

Klausa WHERE! Kita dapat menggunakan WHERE untuk menentukan kondisi pada kolom dan hanya memilih baris yang memenuhi kondisi tersebut. Sebelum masuk ke dalam kueri, ada beberapa kata kunci penting yang perlu kita ketahui.

Klausa WHERE pada SQL dapat dikombinasikan dengan operator AND, OR, dan NOT.

- **AND/OR**

Hal pertama yang perlu diketahui adalah bahwa kita dapat menetapkan beberapa kondisi dalam sebuah query. Secara khusus, ***Condition1 AND Condition2*** berarti sebuah baris harus memenuhi kedua kondisi; ***Condition1***

OR Condition2 berarti sebuah baris hanya perlu memenuhi setidaknya 1 kondisi untuk dipilih.

- **=; >; <; >=; <=**

Ini adalah simbol aritmatika dasar - sama dengan; lebih besar dari; lebih kecil dari; lebih besar dari atau sama dengan; lebih kecil dari atau sama dengan. Kami biasanya menggunakan operator ini pada kolom Numerik. Meskipun mereka juga dapat digunakan pada kolom tipe string/karakter, namun tidak intuitif bagi kita untuk melihat sesuatu dalam urutan abjad.

Operator AND dan OR digunakan untuk catatan filter berdasarkan kondisi yang lebih dari satu.

1. Operator AND menampilkan data jika semua kondisi yang dipisahkan AND bernilai BENAR (TRUE).
2. Operator OR menampilkan data jika salah satu kondisi yang dipisahkan oleh OR bernilai BENAR (TRUE).

Sintaks AND

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 AND condition2 AND condition3 ...;
```

Sintaks OR

```
SELECT column1, column2, ...
FROM table_name
WHERE condition1 OR condition2 OR condition3 ...;
```

Sintaks NOT

```
SELECT column1, column2, ...
FROM table_name
WHERE NOT condition;
```

- **BETWEEN**

BETWEEN memungkinkan kita untuk memilih nilai di antara suatu rentang. Dengan demikian, ini lebih cocok untuk kolom numerik dan nilai waktu, karena tidak ada konsep pengurutan pada kolom kategorikal (misalnya, Adam tidak lebih besar/kecil dari Lily).

Sebagai contoh, kita ingin mencari pelanggan yang usianya antara 20 sampai 40 tahun, kita bisa menggunakan ***WHERE Age BETWEEN 20 AND 40***. Ingatlah bahwa rentang tersebut **bersifat inklusif**, pelanggan yang berusia 20 atau 40 tahun juga akan dipilih.

⚡ Kita dapat mencapai hal yang sama dengan menggunakan operator aritmatika, tetapi menggunakan **BETWEEN** memungkinkan orang lain untuk memahami kueri kita secara efisien. Sebagai ilustrasi, query di atas dapat juga menjadi **WHERE Age >= 20 AND Age <= 40** (Pelanggan harus memenuhi kedua kondisi tersebut untuk dikembalikan).

- **IN**

IN memungkinkan kita untuk menentukan beberapa nilai dan hanya memilih baris dengan nilai-nilai tersebut. Misalkan kita ingin mencari pelanggan yang

tinggal di Paris atau Tokyo, kita bisa menggunakan ***WHERE City IN ('Paris', 'Tokyo')***

Selain itu, jika kita ingin mencari pelanggan yang TIDAK tinggal di Paris atau Tokyo, kita cukup menambahkan kata kunci **NOT sebelum IN** - ***WHERE City NOT IN ('Paris', 'Tokyo')***

- **LIKE**

LIKE memungkinkan kita untuk mencari pola tertentu dalam kolom-kolom yang bertipe Karakter. Meskipun LIKE juga dapat digunakan pada kolom numerik, kita cenderung menggunakan operator aritmatika karena sifat angka.

Untuk mendeskripsikan pola, kita menggunakan **%** atau **_**

1. Dalam SQL, **%** mewakili **0, 1, atau lebih karakter**
2. sedangkan **_** mewakili 1 karakter.

Katakanlah kita ingin menemukan pelanggan yang namanya mengandung ***i***, dan kita tidak peduli apakah nama tersebut diawali dengan ***i***, diakhiri dengan ***i***, atau hanya memiliki ***i*** di tengahnya. Kita dapat menggunakan

WHERE Name LIKE '%i%'

Untuk lebih jelasnya, **%** dapat berupa **0** karakter, sehingga Nama dapat diawali dengan i atau diakhiri dengan i. Atau, % dapat berupa 1 karakter atau lebih, sehingga i dapat berada di tengah-tengah Nama.

References

SQL - Joins

Basic SQL statements you need to know (SELECT/DISTINCT/ORDER BY/LIMIT/WHERE)

Operator AND, OR dan NOT Pada SQL